Chapter 2

Application Layer

Detailed Study Notes with Simple Explanations and Daily-Life Examples

Based on all 88 slides of Chapter_2_v2.pptx
Slides from: Computer Networking: A Top-Down Approach, 9th edition, J.F. Kurose and K.W. Ross, Pearson, 2025

How to use these notes

- The notes follow the **same order as the slides**. Each heading shows the slide numbers it covers.
- Green boxes give a **daily-life example**, blue boxes the **key idea**, amber boxes the **questions asked on the slides** with answers, and red boxes clear **common confusions**.
- Parts marked *extra* go slightly beyond the slides to make a concept clearer.

Table of Contents

Part 1: The Big Picture of the Application Layer 5

1.1 What is this chapter about? 5

1.2 Our goals 6

Part 2: Principles of Network Applications 7

2.1 Some network applications 7

2.2 Creating a network app 7

2.3 The client-server paradigm 8

2.4 Peer-to-peer (P2P) architecture 8

2.5 Processes communicating 9

2.6 Sockets 10

2.7 Addressing processes 10

2.8 What does an application-layer protocol define? 11

2.9 What transport service does an application need? 12

2.10 Internet transport protocol services: TCP and UDP 13

2.11 Which applications use which transport protocol? 14

2.12 Securing TCP 14

Part 3: The Web and HTTP 16

3.1 Web page basics: objects and URLs 16

3.2 HTTP overview 16

3.3 Non-persistent and persistent HTTP 17

3.4 HTTP request message 18

3.5 HTTP response message and status codes 20

3.6 Trying out HTTP yourself 21

3.7 Maintaining state: cookies 21

3.8 Cookies: tracking a user's browsing behaviour 23

3.9 GDPR and cookies 24

3.10 Web caches (proxy servers) 24

3.11 Browser caching: Conditional GET 25

3.12 HTTP/2 26

3.13 From HTTP/2 to HTTP/3 27

3.14 QUIC: Quick UDP Internet Connections 27

Part 4: E-mail: SMTP, IMAP 29

4.1 The three major components of e-mail 29
4.2 SMTP (RFC 5321) 29
4.3 Scenario: Alice sends an e-mail to Bob 30
4.4 Retrieving e-mail: mail access protocols 30

Part 5: The Domain Name System (DNS) 32

5.1 What is DNS and why do we need it? 32
5.2 DNS services and why DNS is not centralized 32
5.3 Thinking about DNS 33
5.4 DNS: a distributed, hierarchical database 33
5.5 Root name servers 34
5.6 TLD servers and authoritative servers 34
5.7 Local DNS name servers 35
5.8 DNS name resolution: iterated vs recursive queries 35
5.9 Caching DNS information 36
5.10 DNS records 37
5.11 DNS protocol messages 37
5.12 Getting your info into the DNS 38
5.13 DNS security 38

Part 6: Video Streaming and Content Distribution Networks 40

6.1 Context: why video is special 40
6.2 Multimedia: video and how it is compressed 40
6.3 Content Distribution Networks (CDNs) 40
6.4 How does Netflix work? 41

Part 7: Socket Programming 42

7.1 Goal and the two socket types 42
7.2 Client/server socket interaction: UDP 42

Part 8: Chapter Summary and Revision Aids 45

8.1 What the chapter covered 45
8.2 Common confusions, cleared 45
8.3 Formulas and numbers to remember 46
8.4 Self-test questions 47

8.5 Glossary 47

Part 1: The Big Picture of the Application Layer

1.1 What is this chapter about?

Slides 1-3, 17, 54, 60, 77

Chapter 2 is about the **Application Layer**, the topmost layer of the Internet's protocol stack. This is the layer that you, as a user and as a programmer, actually touch. Your web browser, your e-mail app, YouTube, Zoom and WhatsApp all live here. Every layer below it (transport, network, link, physical) exists only to carry the application's messages from one device to another.

The five-layer Internet stack (the slides show this on the left of the network picture)

```
+--------------+
| Application  |  Web, e-mail, DNS, streaming   <-- THIS CHAPTER
| Transport    |  TCP, UDP: delivers data from one PROCESS to another
| Network      |  IP: delivers data from one HOST to another
| Link         |  Wi-Fi, Ethernet: delivers data across ONE hop
| Physical     |  actual bits on wire, fibre or air
+--------------+
```

Daily-life example

Imagine you order a gift for a friend in another city. **You** decide what the gift is, what the greeting card says and the friend's address (that is the application layer: the actual purpose of the whole exercise). The **courier company** decides how it is packed and whether it is tracked/insured (transport). The **road and airline network** decides the route (network layer). The truck, the plane and the driver are the lower layers. You never think about the planes, but without them your gift never arrives. In the same way, you never think about routers when you open a web page, but your page needs them.

The roadmap of the chapter

The slide titled *Application layer: overview* keeps reappearing in the deck (slides 2, 17, 54, 60, 77). It is simply a map showing where you are in the chapter. The chapter has these parts:

1. **Principles of network applications** - client-server vs peer-to-peer, processes, sockets, addressing, and what an application needs from the transport layer (slides 4-16).
2. **Web and HTTP** - how web pages are fetched, cookies, caches, HTTP/2, HTTP/3 and QUIC (slides 18-53).
3. **E-mail: SMTP and IMAP** - how mail is sent and retrieved (slides 55-59).
4. **The Domain Name System (DNS)** - how names like www.google.com become IP addresses (slides 61-76).
5. **Video streaming and Content Distribution Networks (CDNs)** - how Netflix and YouTube serve millions of people (slides 78-83).
6. **Socket programming with UDP and TCP** - how you write your own network application (slides 84-86).
7. **P2P applications** (such as BitTorrent) - this appears in the map on slide 77 and in the summary; the idea itself is introduced on slide 7.

1.2 Our goals

Slide 3

The slide lists two kinds of goals:

- **Conceptual and implementation aspects of application-layer protocols.** That means: the ideas (what is a client? what is a server? what does a protocol define?) and the practical side (how do I actually write a program that talks over the network?). The concepts covered are the **transport-layer service models** (what TCP and UDP offer to an application), the **client-server paradigm** and the **peer-to-peer paradigm**.
- **Learning protocols by studying real ones.** Instead of studying protocols in the abstract, we dissect famous ones: HTTP (web), SMTP and IMAP (e-mail), DNS (names), plus video streaming systems and CDNs. Finally we learn **programming network applications** through the **socket API**.

> **Key idea**
>
> A **protocol** is just an agreed set of rules for a conversation between two programs: who speaks first, in what format, what each message means and what to do when something goes wrong. If both sides follow the same protocol they can cooperate even if they were written by different companies in different countries.

Part 2: Principles of Network Applications

2.1 Some network applications

Slide 4

The slide lists examples to show how broad the world of network apps is: social media, the Web, text messaging, e-mail, multi-user network games, streaming stored video (YouTube, Hulu, Netflix), P2P file sharing, voice over IP, real-time video conferencing (for example Zoom), Internet search and remote login. The question at the bottom ("your favorites?") is only an invitation to think of the apps you use daily.

The important observation is that **all of them have the same basic structure**: there are programs running on two (or more) different devices, and these programs exchange messages over the network. What differs between apps is *what* they say to each other, how fast it must arrive and whether anything may be lost. That is exactly why we study protocols and transport requirements next.

2.2 Creating a network app

Slide 5

The big picture on slide 5 shows the Internet: phones, laptops, cars and even a fridge connected through a mobile network, a home network, an enterprise (company) network, a local/regional ISP, a national/global ISP, a content-provider network and a datacenter network. Only some of these devices are drawn with the full stack of five layers (application, transport, network, data link, physical): the **end systems** (your phone, your laptop, the server in the datacenter).

To create a network application you **write programs that:**

- run on different **end systems** (for example, one program on your phone and one on a server), and
- communicate with each other over the network (for example, a web server program talks to a browser program).

You do **not** need to write any software for the devices in the network core (routers and switches). Those devices **do not run user applications**; they only forward data and therefore only implement the lower layers. This is a huge advantage: because applications live only on end systems, a new application can be developed and spread very quickly. You write the server, people install your app, and it works. Nobody has to upgrade the routers of the Internet.

> **Daily-life example**
>
> Think of the road network and delivery businesses. Roads (the network core) just carry vehicles; they do not care whether a vehicle is carrying pizza or books. If you start a new pizza-delivery business you only need a kitchen, a bike and a customer's address (the end systems). You do not have to ask the government to rebuild any road. This is exactly why new apps such as TikTok could reach the whole world in a few years.

2.3 The client-server paradigm

Slide 6

In the **client-server architecture** there are two roles:

The server

- It is an **always-on host**: it must be up and running all the time, waiting for requests.
- It has a **permanent IP address**, so clients always know where to find it.
- It is often placed in **data centers, for scaling**: one machine cannot serve millions of people, so a huge number of servers are grouped together.

The clients

- They **contact and communicate with the server** (they start the conversation).
- They **may be intermittently connected**: your laptop is switched off at night and that is fine.
- They **may have dynamic IP addresses**: the address can be different every time you connect.
- They **do not communicate directly with each other**. If two clients want to exchange something they must go through the server.

Examples of client-server protocols given on the slide: **HTTP, IMAP and FTP**.

Daily-life example

A restaurant is a perfect client-server system. The restaurant (server) is open at fixed hours, at a **fixed address** that everyone knows (permanent IP). The customers (clients) arrive and leave whenever they like and sit at different tables each visit (dynamic address). Customers talk only to the waiter, never directly to the kitchen or to customers at other tables. On Eid day when many customers arrive, the restaurant must have more cooks and tables, which is why large services keep hundreds of servers in data centers.

Key idea

Weak points of client-server: if the server is down nobody gets service (single point of failure), and every new user adds load on the same server, so the owner must keep buying capacity.

2.4 Peer-to-peer (P2P) architecture

Slide 7

In a **peer-to-peer** architecture there is **no always-on server** (or the server plays only a small role). Arbitrary end systems, called **peers**, communicate **directly** with each other. Every peer both **requests service from other peers and provides service in return** to other peers.

- **Self scalability:** when a new peer joins, it brings new *demand* (it wants files) but also new *capacity* (it can upload to others). So the system grows its own capacity as the number of users grows.
- **Peers are intermittently connected and change IP addresses.** Because nobody is permanently online at a fixed address, it is difficult to track who has what.
- **Complex management** is therefore the price you pay.

- Example: **BitTorrent** file sharing.

Daily-life example

A group of classmates before an exam. If only one student has the notes and everybody must borrow them from him, he becomes the bottleneck (client-server). Instead, suppose each student photocopies different chapters and swaps pages with others. Now, the more students there are, the more people who hold pieces and can hand them out. That is self scalability. The disadvantage is that nobody is in charge: students come and go, and you must keep track of who holds which chapter.

Client-server vs P2P at a glance

- **Server:** client-server has one (or a few) always-on server; P2P has none.
- **Who serves whom:** in client-server only the server serves; in P2P everybody serves everybody.
- **Scalability:** client-server needs more servers as users increase; P2P scales itself.
- **Management:** client-server is simple and centrally controlled; P2P is complex.

2.5 Processes communicating

Slide 8

Strictly speaking it is not *computers* that communicate; it is **processes**. A **process** is simply a program that is currently running inside a host (for example, the Chrome browser that you opened is a process; a web server software running is another process).

- Two processes **within the same host** communicate using **inter-process communication (IPC)**, which is defined and provided by the operating system.
- Two processes **on different hosts** communicate by **exchanging messages** across the network. This is what the application layer is about.

When two processes talk we give them names according to their role in the conversation:

- **Client process:** the process that **initiates** the communication.
- **Server process:** the process that **waits to be contacted**.

Common confusion

Client and server are **roles of processes**, not types of machines. The same computer can run a browser (client process) and a web server (server process) at the same time. In P2P applications a single peer runs **both** a client process (when it downloads) and a server process (when others download from it). The slide says this in its last line.

Daily-life example

Two people in the same house can simply shout to each other or tap on the shoulder (inter-process communication, handled by 'the house itself', the OS). If they live in different cities they must send letters (messages over the network). The person who starts the conversation by writing the first letter plays the client; the one who sits and waits for letters plays the server.

2.6 Sockets

Slide 9 (and again slide 84)

A process does not write directly onto the wires. It sends and receives its messages through an interface called a **socket**. The slide gives the best analogy: **a socket is like a door**.

- The sending process pushes the message **out of its door**.
- It relies on the **transport infrastructure on the other side of the door** to carry the message across the Internet and to deliver it to the **door (socket) of the receiving process**.
- So **two sockets are involved, one on each side**.

Look at the figure on the slide: in each host there is an application layer containing the process, then the socket, and below the socket the transport, network, link and physical layers. The labels explain who is responsible for what:

- Everything on the **process side of the socket is controlled by the application developer** (you write the program, decide what the messages look like, and choose which transport protocol, TCP or UDP, to use).
- Everything **below the socket is controlled by the operating system** (how packets are formed, routed and retransmitted is not your job).

Daily-life example

A socket is exactly like the front door (or the letterbox) of your house. When you want to send a letter you only walk to the letterbox and drop it in; you do not care how the postal service sorts, flies and drives it. At the other end, the post office puts the letter into the receiver's letterbox, and he picks it up from there. You only control what you put in the box and what you do with what you find in it. The postal service (the OS and the network) controls everything in between.

2.7 Addressing processes

Slide 10

To receive messages, a process must have an **identifier**, otherwise nobody can say who the message is for. Every host has a unique **32-bit IP address** (IPv4), for example 128.119.245.12 (four numbers, each of 8 bits).

Question asked on the slide

Q: Does the IP address of the host on which a process runs suffice to identify the process?

A: No. Many processes can be running on the same host at the same time (a browser, a mail client, a music app, a game...). The IP address finds the host; it does not say which program on it should receive the message.

Therefore the identifier of a process includes **both the IP address and a port number**. Popular server programs have well-known port numbers:

- HTTP web server: port **80**
- Mail server (SMTP): port **25**

Example from the slide: to send an HTTP message to the web server gaia.cs.umass.edu you must address it to **IP address 128.119.245.12, port number 80**.

Daily-life example

An apartment building. The **street address** of the building is the IP address: it gets the courier to the right building. But dozens of families live there, so the parcel also needs the **flat number**: that is the port number. Port 80 is like 'flat number 80 is always the reception desk', a well-known flat that everyone knows how to reach.

Extra note (not on the slide): port numbers are 16-bit numbers (0 to 65535). Port numbers up to 1023 are reserved for well-known services; your own programs should use bigger ones, for example 12000 in the socket examples later.

2.8 What does an application-layer protocol define?

Slide 11

A protocol is the rulebook of a conversation. An application-layer protocol defines four things:

1. **Types of messages exchanged**, for example *request* and *response*.
2. **Message syntax**: which *fields* a message has and how the fields are separated (delineated) from each other.
3. **Message semantics**: the *meaning* of the information in each field.
4. **Rules** for *when* and *how* a process sends a message and how it responds to one.

Daily-life example

A telephone call has a protocol too, even though we never write it down. **Message types:** greeting, question, answer, goodbye. **Syntax:** we speak in a language, in sentences, one after another. **Semantics:** 'Hello?' means 'I picked up the phone, who is there?'. **Rules:** the person who dialled waits for the other to say hello first, we take turns in speaking, and we say goodbye before hanging up. If someone broke these rules the call would be confusing, and in the same way two programs break down if they disagree about the protocol.

Open vs proprietary protocols

- **Open protocols** are defined in documents called **RFCs** (Requests for Comments) which **everyone can read**. This allows **interoperability**: Chrome, Firefox and Safari, written by different companies, can all talk to any web server in the world because HTTP is open. Examples: HTTP, SMTP.
- **Proprietary protocols** are owned by a company and are not publicly specified. Example from the slide: **Zoom**. Only Zoom's own software is guaranteed to understand it.

Common confusion

Do not mix up an **application** with its **protocol**. The Web is an application; HTTP is the application-layer protocol of the Web. E-mail is an application; SMTP and IMAP are its protocols. The protocol is only one piece of the application (the web also includes HTML, browsers, servers).

2.9 What transport service does an application need?

Slides 12-13

Different applications need different things from the network below them. The slide groups the needs into four dimensions. Learn these four words well because they are used to compare applications and to choose between TCP and UDP.

1) Throughput

Throughput is how many bits per second get through. Some applications, such as multimedia, require a **minimum amount of throughput** to be 'effective' (video that needs 3 Mbps is useless at 500 kbps). Other applications are called **elastic applications**: they use whatever throughput they get, and simply finish sooner when more is available (e.g., a file download).

> **Daily-life example**
>
> Water in a pipe. A shower needs a minimum flow or it is no good (needs minimum throughput). Filling a bucket works with a trickle or with a strong flow: it just takes longer or shorter (elastic).

2) Data integrity (data loss)

Some applications, such as **file transfer and web transactions, need 100% reliable data transfer**: one lost byte may corrupt a program or change a bank balance. Other applications, such as **audio, can tolerate some loss**.

> **Daily-life example**
>
> If a word is lost in a phone call you still understand the sentence (loss-tolerant). But if one digit of an account number is lost when you transfer money, it is a disaster (no loss allowed).

3) Timing

Some applications, such as **Internet telephony and interactive games, require low delay** to be effective. If your voice reaches the other person two seconds late, the conversation becomes unusable even if every bit arrives correctly.

> **Daily-life example**
>
> A live cricket commentary must arrive in real time, otherwise your neighbour shouts 'six!' before you see it. Downloading the scorecard PDF after the match, however, can take as long as it likes.

4) Security

Applications may require **encryption** (so that eavesdroppers cannot read the data) and **data integrity** (so that nobody can alter the data on the way).

What common applications need (slide 13, explained in words)

- **File transfer / download:** throughput elastic; no data loss allowed; not time sensitive.
- **E-mail:** elastic; no loss; not time sensitive (a mail arriving 10 seconds later hurts nobody).
- **Web documents:** elastic; no loss; not time sensitive (though people hate waiting, there is no hard timing requirement).

- **Real-time audio/video** (a video call): audio needs about 5 Kbps - 1 Mbps and video 10 Kbps - 5 Mbps; loss tolerant; very time sensitive (delay in the tens of milliseconds).
- **Streaming audio/video** (YouTube): same throughput as above; loss tolerant; time sensitive only to the order of a few seconds, because the player stores some seconds of video in advance (buffering).
- **Interactive games:** a few Kbps or more; loss tolerant; time sensitive in tens of milliseconds.
- **Text messaging:** elastic; no loss (you cannot lose half a sentence); 'yes and no' for timing: you want it quickly but a small delay is acceptable.

Key idea

Real-time and streaming apps care about **speed and timing**; file transfer, e-mail and web pages care about **correctness**. This single sentence explains most of the TCP vs UDP choices below.

2.10 Internet transport protocol services: TCP and UDP

Slide 14

The Internet offers applications two transport protocols. The application developer picks one when creating the socket.

TCP service (Transmission Control Protocol)

- **Reliable transport** between sending and receiving process: data arrives complete, uncorrupted and in order.
- **Flow control:** the sender will not overwhelm the receiver.
- **Congestion control:** it throttles (slows down) the sender when the network is overloaded.
- **Connection-oriented:** a setup (handshake) between client and server processes is required before data flows.
- **Does not provide:** timing guarantees, a minimum throughput guarantee, or security.

UDP service (User Datagram Protocol)

- **Unreliable data transfer** between sending and receiving process: messages may be lost or arrive out of order and UDP will not fix it.
- **Does not provide:** reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

These sound like a long list of what is missing, which leads to the slide's question:

Question asked on the slide

Q: Why bother? Why is there a UDP?

A: Because UDP is **fast and simple**. There is no connection setup delay, no waiting for acknowledgements, no slowing down because of congestion control, and very little header overhead. For applications that can tolerate some loss but are sensitive to delay (voice calls, live video, games) this is ideal. UDP is also a bare foundation on which a designer can build his own custom features, which is exactly what QUIC does later in the chapter.

The terms explained simply

- **Reliable transfer** = registered courier with a delivery receipt. If the parcel is lost, it is sent again. UDP is an ordinary postcard: dropped in the box and no one guarantees anything.
- **Flow control** protects the *receiver*. Pouring tea from a big jug into a small cup: you pour slowly so that the cup does not overflow. The receiver's buffer is the cup.
- **Congestion control** protects the *network*. When a road is jammed, drivers slow down. If everybody kept rushing in, the jam would get worse and nobody would reach. TCP senders slow down when they sense the network is overloaded.
- **Connection-oriented** = phone call. You dial, the other side picks up and says hello, and only then do you talk. UDP is more like dropping a letter without arranging anything first.

> **Daily-life example**
>
> Live match streaming (UDP-like): if one tiny piece of the picture is lost, the player simply skips it. Asking the server to resend that old piece would make you fall behind the live action, which is worse than a tiny glitch. Downloading a PDF (TCP): a lost piece must be repeated, otherwise the file is corrupt and cannot be opened.

2.11 Which applications use which transport protocol?

Slide 15

- **File transfer/download:** FTP (RFC 959) over **TCP**.
- **E-mail:** SMTP (RFC 5321) over **TCP**.
- **Web documents:** HTTP (RFC 7230, 9110) over **TCP**.
- **Internet telephony:** SIP (RFC 3261), RTP (RFC 3550) or a proprietary protocol; over **TCP or UDP**.
- **Streaming audio/video:** HTTP (RFC 7230) and DASH over **TCP**.
- **Interactive games:** proprietary protocols such as those of World of Warcraft (WOW) and first-person shooters (FPS); over **UDP or TCP**.

The pattern: applications that cannot afford any loss (files, mail, web pages) use TCP. Real-time applications that care about delay may choose UDP. Notice that streaming video, although time-sensitive, uses TCP: since it only needs to be on time within a few seconds, buffering lets it enjoy TCP's reliability. (DASH, mentioned on the slide, is Dynamic Adaptive Streaming over HTTP: the video player requests the video in small chunks over HTTP and picks the quality that suits the current network speed. It returns in the Netflix part of these notes.)

2.12 Securing TCP

Slide 16

Plain TCP and UDP sockets (called **vanilla** sockets) have **no encryption**. If you type a password and send it into such a socket, it travels across the Internet in **cleartext**: any machine along the path could read it. The slide marks this with an exclamation (!).

The fix is **Transport Layer Security (TLS)**. It provides:

- **Encrypted TCP connections** (nobody on the path can read the data),

- **Data integrity** (nobody can change the data without being noticed),
- **End-point authentication** (you can be sure you are really talking to the real bank, not an impostor).

An important detail: **TLS is implemented in the application layer**. Applications use *TLS libraries*, and these libraries use TCP in turn. The application hands cleartext to the TLS library ('cleartext sent into socket'), the library encrypts it, and what actually **traverses the Internet is encrypted**. The details come in Chapter 8. (HTTP running over TLS is what we call HTTPS.)

> **Daily-life example**
>
> A normal TCP socket is like sending your ATM PIN on an open **postcard**: the postman, the sorting clerk and anyone curious can read it. TLS is a **sealed, tamper-evident envelope** that only the real recipient can open, and which also carries proof that the recipient is who he claims to be.

Part 3: The Web and HTTP

3.1 Web page basics: objects and URLs

Slide 18

A web page is not a single file. It consists of many **objects**, and each object can be stored on a **different web server**. An object can be an HTML file, a JPEG image, a Java applet, an audio file and so on. Most pages have a **base HTML file** that includes references to several other objects. Each object is addressable by a **URL** (Uniform Resource Locator), for example:

```
www.someschool.edu/someDept/pic.gif
|_________________| |_____________|
    host name          path name
```

- **Host name** (www.someschool.edu): which server holds the object.
- **Path name** (/someDept/pic.gif): where the object sits inside that server.

> **Daily-life example**
>
> A newspaper page. The page layout (HTML) is a sheet of instructions: 'headline here, photo number 1 here, advertisement there'. The photos and advertisements are separate items that may come from different photographers and companies. Your browser first receives the instruction sheet, reads it, and then goes to fetch each photo separately using its URL (its full address).

3.2 HTTP overview

Slides 19-20

HTTP stands for **HyperText Transfer Protocol**. It is the Web's application-layer protocol and follows the **client-server model**:

- **Client:** the browser (Firefox on a PC, Safari on an iPhone). It **requests, receives and displays** Web objects, using HTTP.
- **Server:** the Web server (for example Apache). It **sends objects in response to requests**.

The browser sends an **HTTP request**, and the server answers with an **HTTP response**. That is the whole idea.

HTTP uses TCP

1. The client **initiates a TCP connection** (creates a socket) to the server, on **port 80**.
2. The server **accepts** the TCP connection from the client.
3. **HTTP messages** (application-layer protocol messages) are exchanged between the browser (HTTP client) and the Web server (HTTP server).
4. The **TCP connection is closed**.

HTTP is 'stateless'

The server maintains **no information about past client requests**. Every request is treated as brand new. The 'aside' on the slide explains why the designers chose this: protocols that maintain state are **complex**. Past history (state) must be stored, and if the server or the client crashes, their views of the state may become inconsistent and must be reconciled.

> **Daily-life example**
>
> Imagine a shopkeeper who has no memory at all. Each time you step to the counter he treats you as a complete stranger, so you must say your whole request each time ('one packet of biscuits, please'). That is a stateless server. It is simple and he never gets confused, but he cannot remember your earlier purchases. A stateful shopkeeper, who remembers 'this customer is in the middle of buying a wedding set', must keep notes and, if he suddenly falls ill halfway through, nobody knows what was already agreed.

3.3 Non-persistent and persistent HTTP

Slides 21-25

There are two ways to use TCP connections for HTTP:

- **Non-persistent HTTP:** a TCP connection is opened, **at most one object** is sent over it, and then the connection is closed. Downloading multiple objects therefore needs **multiple connections**.
- **Persistent HTTP:** a TCP connection is opened to a server, **multiple objects can be sent over that single connection** between the client and that server, and only then it is closed.

Non-persistent HTTP: the step-by-step example (slides 22-23)

A user types the URL **www.someSchool.edu/someDepartment/home.index**. This page contains text and **references to 10 JPEG images**.

1. **1a.** The HTTP client starts a TCP connection to the HTTP server at www.someSchool.edu on port 80. **1b.** The server, which was waiting at port 80, **accepts** the connection and tells the client.
2. The client sends an **HTTP request message** (containing the URL) into the TCP connection socket. It says it wants the object someDepartment/home.index.
3. The server receives the request, forms a **response message** containing the requested object, and sends it into its socket.
4. The server **closes** the TCP connection.
5. The client receives the response containing the HTML file and displays it. While parsing the file, it finds **10 referenced JPEG objects**.
6. Steps 1-5 are **repeated for each of the 10 JPEG objects**.

So with non-persistent HTTP this page needs **11 separate TCP connections** (1 for the HTML file + 10 for images).

Response time: RTT

RTT (Round-Trip Time) is the time for a small packet to travel from the client to the server **and back**.

Daily-life example

Throw a ball at a wall and catch it when it bounces back. The time from throwing to catching is the RTT. If a friend lives across the country, the RTT of a phone call 'Hello?' - 'Hello!' is noticeably longer than for a friend next door.

For non-persistent HTTP, the response time **per object** consists of:

1. **One RTT to initiate the TCP connection** (the handshake: 'can you hear me?' - 'yes').
2. **One RTT for the HTTP request** and the first few bytes of the HTTP response to come back.
3. The **file transmission time** (time to push all the bytes of the object).

Key idea

Non-persistent HTTP response time = 2 RTT + file transmission time (per object).

Persistent HTTP (HTTP/1.1)

The slide lists the problems of non-persistent HTTP:

- It needs **2 RTTs per object**.
- The operating system has **overhead for each TCP connection** (memory, buffers, state).
- Browsers therefore often open **multiple parallel TCP connections** to fetch the referenced objects in parallel, which is a partial fix but adds even more overhead.

Persistent HTTP (HTTP/1.1) solves this: the **server leaves the connection open** after sending a response. Subsequent HTTP messages between the same client and server are sent over the open connection. The client sends a request **as soon as it encounters a referenced object**. In the best case, as little as **one RTT for all the referenced objects**, which cuts the response time roughly in half.

Daily-life example

Worked numbers. Take a page with 1 HTML file and 10 images, an RTT of 100 ms, and ignore the transmission time to keep it simple. **Non-persistent:** 11 objects x 2 RTT = 22 RTT = **2200 ms** (more than 2 seconds!). **Persistent:** 1 RTT for the TCP handshake + 1 RTT for the HTML file + about 1 RTT for all the 10 images requested one after another = about 3 RTT = **300 ms**.

Daily-life example

A grocery run. Non-persistent: for every single item on your list you call a new taxi, ride to the shop, buy one item, and the taxi leaves; you repeat this 11 times. Persistent: you call one taxi, keep it waiting outside the shop, and fill all your items into it one by one.

3.4 HTTP request message

Slides 26-28

There are two kinds of HTTP messages: **request** and **response**. HTTP messages are written in **ASCII**, which means they are human-readable text. A request looks like this (slide 26):

A real HTTP GET request

```
GET /index.html HTTP/1.1\r\n                                        <- request line
Host: www-net.cs.umass.edu\r\n                                     <- header lines
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) ...\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n                                                  <- blank line = end of headers
```

- **Request line:** contains the **method** (GET, POST, HEAD, ...), the **URL** (the path of the wanted object) and the **HTTP version**.
- **Header lines:** extra information, one per line, of the form *name: value*.
- **\r\n** means **carriage return** and **line feed** (the old typewriter way of saying 'new line'). Each line ends with it. An **empty line** (a carriage return and line feed at the start of a line) signals the **end of the header lines**.

What the headers in the example mean

- **Host:** the host name of the server. It is needed because one server (one IP address) may host many websites, and the server must know which one you want.
- **User-Agent:** which browser/operating system is making the request (here Firefox on a Mac).
- **Accept:** which types of content the client can handle (HTML here).
- **Accept-Language:** preferred language (English).
- **Accept-Encoding:** compression formats the client understands (gzip, deflate), so that the server can send a compressed, smaller file.
- **Connection: keep-alive:** 'please do not close the connection after this response': that is the persistent HTTP of the previous section.

General format of a request message (slide 27)

The figure on slide 27 shows the same thing in generic form. **sp** means a single space, **cr** a carriage return and **lf** a line feed:

```
method <sp> URL <sp> version <cr><lf>              <- request line
header field name: value <cr><lf>                  <- header lines (as many as needed)
...
header field name: value <cr><lf>
<cr><lf>                                           <- blank line ends the headers
entity body                                        <- (optional) data for server
```

Other request methods (slide 28)

- **GET:** asks the server for an object. Data can also be sent to the server inside the URL after a '?', e.g., **www.somesite.com/animalsearch?monkeys&banana**.
- **POST:** used when a web page includes a **form**. The user's input is sent in the **entity body** of the request.
- **HEAD:** asks the server for the **headers only**, the ones that would be returned if the URL were requested with GET. No object is returned.

- **PUT: uploads** a new file (object) to the server and **completely replaces** the file at the specified URL with the content in the entity body. (The slide's wording says 'POST' here; it is the PUT request's entity body.)

Daily-life example

Think of a librarian. **GET**: 'Please bring me that book.' **POST**: you fill out and hand over a membership form. **HEAD**: 'Do you have that book, and when was it last updated?' - he reads out the details from the cover but does not bring the book. **PUT**: you hand him a new edition, and he removes the old copy and puts yours in its place.

Common confusion

GET can also carry data (in the URL) but this is visible in the address bar and is limited in length. Forms with a password should use POST (and, of course, TLS).

3.5 HTTP response message and status codes

Slides 29-30

A real HTTP response

```
HTTP/1.1 200 OK                                       <- status line
Date: Tue, 08 Sep 2020 00:53:20 GMT                   <- header lines
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.9 ...
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...                          <- the requested object
```

- **Status line:** protocol version, **status code** and **status phrase**.
- **Date:** when the response was created. **Server:** the server software.
- **Last-Modified:** when the object was last changed (this is used by caching, see conditional GET).
- **ETag:** a label for the particular version of the object. **Accept-Ranges:** says whether the server can send parts of a file.
- **Content-Length:** size of the object in bytes. **Content-Type:** what kind of data it is (here HTML text).
- **Data:** the actual requested object, after a blank line.

Sample status codes (appear in the first line of the response)

- **200 OK** - the request succeeded, the requested object is later in this message.
- **301 Moved Permanently** - the object has moved; the new location is specified later in this message in the *Location:* field.
- **400 Bad Request** - the server did not understand the request message.
- **404 Not Found** - the requested document is not on this server.

- **505 HTTP Version Not Supported** - the server does not support the HTTP version of the request.

> **Daily-life example**
>
> A shop answering you. 200: 'Here you go.' 301: 'We have shifted to a new address; go there.' 400: 'Sorry, I did not understand what you asked.' 404: 'We do not have that item.' 505: 'We do not speak the language/version you are using.' The first digit tells the story: 2xx success, 3xx redirection, 4xx client error, 5xx server error.

3.6 Trying out HTTP yourself

Slide 31

The best way to understand HTTP is to type a request by hand. The slide's recipe:

1. Use **netcat** to open a TCP connection to port 80 of a web server (gaia.cs.umass.edu). Anything you type is now sent to port 80 of that server.
2. Type a minimal but complete GET request: *GET /kurose_ross/interactive/index.php HTTP/1.1* then *Host: gaia.cs.umass.edu*, and press Enter twice (the blank line ends the headers).
3. Look at the response message that the server sends back. (Alternatively use Wireshark to look at the captured request/response.)

The slide shows the macOS and Windows commands. On EndeavourOS (Arch) you can do it like this:

```
# install the OpenBSD netcat and Wireshark (only once)
sudo pacman -S openbsd-netcat wireshark-qt

# open the TCP connection; -C sends proper carriage-return/line-feed endings
nc -C gaia.cs.umass.edu 80
# now type these two lines, then press Enter twice
GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu

# easier alternative: curl shows both request and response headers
curl -v http://gaia.cs.umass.edu/kurose_ross/interactive/index.php

# HEAD request: headers only
curl -I http://gaia.cs.umass.edu/kurose_ross/interactive/index.php
```

3.7 Maintaining state: cookies

Slides 32-35

Why do we need state at all?

Recall that HTTP is stateless: every HTTP GET/response is independent. There is **no notion of a multi-step exchange** and the client and server need not **recover** from a transaction that was only partially completed. That keeps HTTP simple.

Slide 32 contrasts it with a **stateful protocol**, in which the client makes **two changes to X, or none at all**. The picture is a transaction: the client asks to **lock** data record X, the server replies OK, the client sends **update X to X'** (OK), then **update X' to X''** (OK), then **unlock X**. The two updates must

both happen or neither must happen.

Question asked on the slide

Q: What happens if the network connection or the client crashes at time t' (between the two updates)?

A: The server is left with a record that is locked and half-updated (X' instead of X or X''). It must detect the failure, undo or finish the partial transaction and release the lock, so both sides need complicated recovery logic. This is why stateful protocols are complex and why HTTP chose to be stateless.

But real websites need to remember you (your login, your shopping cart). The solution is **cookies**: websites and the client browser use cookies to maintain **some state between transactions** without making HTTP itself stateful.

The four components of cookie technology

1. A **cookie header line in the HTTP response message** (the server says: remember this number).
2. A **cookie header line in the next HTTP request message** (the browser says: here is the number you gave me).
3. A **cookie file kept on the user's host** and managed by the user's browser.
4. A **back-end database at the website**.

Example: Susan visits an e-commerce site (slides 33-34)

Susan uses a browser on her laptop and visits a specific e-commerce site (Amazon in the figure) **for the first time**.

1. Her browser sends the usual HTTP request message. It contains no cookie for amazon.
2. When the request arrives, the site creates a **unique ID** (here 1678) and an **entry in its back-end database** for that ID.
3. The site's HTTP response includes the header **Set-cookie: 1678**.
4. Susan's browser **appends a line to its cookie file**: amazon 1678 (the file already holds other sites' cookies, e.g., ebay 8734).
5. On every **subsequent request** to that site, the browser adds **Cookie: 1678**. The site uses the ID to look her up in the database and performs **cookie-specific actions** (show her cart, keep her logged in, ...).
6. **One week later**, she comes back: the cookie file still has amazon 1678, so the site recognizes her.

Key idea

The cookie itself is normally just an **ID number**. All the real information about you (the cart, the history) lives in the **website's database**. The browser only carries the number around.

Daily-life example

A **loyalty card** or a **cloakroom token**. The first time you hand your coat to the cloakroom you get token number 1678. The cloakroom keeps your coat on shelf 1678 and does not remember your face at all. Whenever you come back and show token 1678 the attendant instantly knows which coat is yours. HTTP stays 'stateless' (the attendant does not remember faces), yet you get personal service because you carry the token.

What cookies are used for, and privacy

- **Authorization** (staying logged in), **shopping carts**, **recommendations**, and **user session state** (such as Web e-mail).
- **Privacy concern:** cookies permit sites to learn a lot about you on their site. And **third-party persistent cookies (tracking cookies)** allow a common identity (cookie value) to be tracked across multiple websites (explained next).

The slide ends with the challenge **'How to keep state?'** and two answers: (1) **at protocol endpoints**: maintain state at the sender/receiver over multiple transactions; (2) **in messages**: cookies in HTTP messages carry the state.

3.8 Cookies: tracking a user's browsing behaviour

Slides 36-40

First, how a page is assembled (slide 36)

When you display a New York Times page: (1) your browser sends **HTTP GET for the base HTML file** to nytimes.com, (2) nytimes.com sends the HTTP reply. The base file refers to an advertisement stored on another site, so the browser (4) **fetches the ad from AdX.com** and (5) gets the ad back, and finally **displays the composed page**: the NY Times page with the embedded ad. So one page visit contacts **two different organizations**.

First-party and third-party cookies (slides 37-39)

- A **first-party cookie** comes from the website you **chose to visit** (the one that provides the base HTML file). In the example, nytimes.com sends **Set cookie: 1634** and records '1634: sports, 2/15/22' in its own database.
- A **third-party cookie** comes from a website you did **not choose to visit**. When your browser fetches the ad from AdX.com, AdX also sends **Set cookie: 7493**, and the request contains **Referrer: NY Times Sports**, so AdX records '7493: NY Times sports, 2/15/22'.

Now follow the story of the user over several days:

1. **Day 1 (2/15/22):** The user reads NY Times sports. nytimes.com has cookie 1634 for him; AdX.com has cookie 7493 for him and knows that 7493 visited NY Times sports.
2. **Day 2 (2/16/22):** The user visits **socks.com**, which also shows ads provided by AdX. The browser's request to AdX carries **cookie: 7493** and **Referrer: socks.com**. AdX adds the record '7493: socks.com, 2/16/22'. AdX now knows that the same person was interested in NY Times sports and in socks.
3. **Day 3 (2/17/22), one day later:** The user visits the NY Times arts section. The browser again sends cookie 7493 to AdX, so AdX recognizes him, remembers he looked at socks, and **returns**

an advertisement for socks inside the arts page.

> **Daily-life example**
> You walk into different shops in a big market. In each shop there is a poster from the same advertising company, and a hidden employee of that company notes down: 'Customer with badge no. 7493 entered the sports shop, then the sock shop'. Next day when you visit the art gallery (which also has this company's poster), the poster suddenly shows socks. You never visited the advertising company; its poster just sat inside the shops you visited.

Summary of cookie tracking (slide 40)

- Cookies can **track user behaviour on a given website** (first-party cookies).
- They can **track user behaviour across multiple websites** (third-party cookies) **without the user ever choosing to visit the tracker site**.
- Tracking **may be invisible**: instead of a visible ad triggering the HTTP GET to the tracker, it could be an **invisible link** (a 1x1 pixel, for instance).
- Third-party tracking via cookies is **disabled by default in Firefox and Safari**. The slide says it was **to be disabled in Chrome in 2023** (in practice this plan has been delayed several times, so for the exam just quote the slide).

3.9 GDPR and cookies

Slide 41

The **GDPR (EU General Data Protection Regulation)** is a European law from May 2018. In relation to cookies it says that the **user has explicit control over whether or not cookies are allowed**. That is why most websites show a 'cookie consent' banner. The quoted **Recital 30** explains the reasoning: online identifiers (IP addresses, cookie identifiers, etc.) leave traces which, especially when combined with unique identifiers and other information received by the servers, can be used to create profiles of people and to identify them.

> **Key idea**
> When cookies can identify an individual, they are considered **personal data** and are subject to GDPR personal data regulations.

3.10 Web caches (proxy servers)

Slides 42-43

The goal of a **Web cache** is to **satisfy the client's request without involving the origin server** (the original server that owns the object). It works like this:

1. The user **configures the browser to point to a (local) Web cache**.
2. The browser sends **all its HTTP requests to the cache**.
3. **If the object is in the cache**, the cache returns the object to the client straight away.
4. **Otherwise** the cache requests the object from the origin server, **stores a copy** of what it received, and then returns the object to the client.

A Web cache is also called a **proxy server**. It acts as **both a client and a server**: it is a **server** for the original requesting client, and a **client** to the origin server.

Why do Web caching?

- **Reduce the response time** for client requests, because the cache is **closer to the client**.
- **Reduce traffic on an institution's access link** (the single, often expensive, link from, say, a university to the Internet). If 40 out of 100 requests are answered by the cache, the access link carries about 40% less traffic.
- The Internet is **dense with caches**, which enables 'poor' content providers to deliver content effectively too. A small website with one slow server benefits because many requests never reach it. (The slide's note: like in real estate, *location is everything*; it is best to have the content close to the clients.)

How does the cache know whether it may store an object and for how long? The **server tells the cache about the object's allowable caching in a response header** (for example, Cache-Control lines saying how long the copy may be kept).

Daily-life example

A **neighbourhood grocery shop**. Instead of every household driving to the big wholesale market (the origin server) far away, the local shop buys popular items in bulk. If you ask for rice and he has it, you get it immediately (cache hit). If not, he fetches it from the wholesale market, gives it to you and also keeps some on the shelf for the next customer (cache miss, then store). The shop acts as a *customer* towards the wholesaler and as a *seller* towards you; that is the 'client and server at the same time' of the slide.

3.11 Browser caching: Conditional GET

Slide 44

Even when the client has a cached copy of an object, the object on the server might have changed. Sending the object again wastes time if it did not change. The **conditional GET** avoids that. The **goal**: do not send the object if the browser has an up-to-date cached version. That means **no object transmission delay** and no use of network resources.

- **Client:** specifies the **date of its cached copy** in the HTTP request: **If-modified-since: <date>**.
- **Server:** if the object was **not modified** after that date, the response contains **no object** - only the status line **HTTP/1.0 304 Not Modified** (the figure's upper half).
- If the object **was modified** after that date, the server replies **200 OK** with the new data (the figure's lower half).

Daily-life example

You own last year's edition of a textbook. You ask the bookshop, 'Has a newer edition been published since June 2025?' If the answer is 'No, nothing new' (304 Not Modified), you keep using your own copy and no book changes hands. If it is 'Yes', he gives you the new edition (200 OK with the data). A one-line question and answer is far cheaper than buying the book again every time.

3.12 HTTP/2

Slides 45-48

The **key goal of HTTP/2** is **decreased delay in multi-object HTTP requests** (pages with many objects).

The problem in HTTP/1.1 (slide 45)

- HTTP/1.1 introduced **multiple, pipelined GETs over a single TCP connection**.
- But the server responds **in order** to the GET requests, using **FCFS (first-come-first-served)** scheduling.
- With FCFS, a **small object may have to wait** for the transmission of a **large object** that was requested earlier. This is called **head-of-line (HOL) blocking**.
- Also, **loss recovery** (retransmitting lost TCP segments) **stalls the transmission of objects**.

> **Daily-life example**
>
> A **supermarket checkout** with one cashier and one queue. You hold a single bottle of milk, but the person in front of you has a trolley full of items. You must wait until all his items are scanned even though your job takes five seconds. The person at the head of the line blocks everyone behind: head-of-line blocking.

What HTTP/2 changes (slide 46)

HTTP/2 (RFC 7540, 2015) gives the server **more flexibility in sending objects to the client**. The methods, status codes and most header fields are **unchanged** from HTTP/1.1. The new features:

- The **transmission order** of requested objects is based on a **client-specified object priority**, not necessarily FCFS.
- The server can **push unrequested objects** to the client (for example, it knows that a page will need a CSS file and sends it before being asked).
- **Objects are divided into frames**, and the frames are **scheduled (interleaved)** to mitigate HOL blocking.

The example in the figures (slides 47-48)

The client requests one **large object O1** (for example a video file) and three **small objects O2, O3, O4**, in this order.

- **HTTP/1.1:** objects are delivered in the order requested. O2, O3 and O4 **wait behind O1**.
- **HTTP/2:** each object is split into **frames** and the frames are **interleaved** during transmission. O2, O3 and O4 are delivered quickly, and O1 is only slightly delayed.

> **Key idea**
>
> Slide notes: this is like **shortest job first**, which **decreases the average delay**.

Daily-life example

Numbers. Suppose O1 needs 10 time units and each small object only 1 unit. HTTP/1.1 (in order): O1 finishes at 10, O2 at 11, O3 at 12, O4 at 13, so the average completion time is (10+11+12+13)/4 = **11.5**. HTTP/2 (one frame of each in turn): O2 finishes at about 2, O3 at 3, O4 at 4 and O1 still finishes at 13, average = (2+3+4+13)/4 = **5.5**. The large file loses almost nothing, but the small ones gain a lot.

Back at the checkout: the cashier now scans a few items for the first person, then your milk, then continues with the trolley. You leave in a minute, and the trolley person is only slightly delayed.

3.13 From HTTP/2 to HTTP/3

Slide 49

HTTP/2 still runs over a **single TCP connection**, and that brings remaining problems:

- **Recovery from packet loss still stalls all object transmissions.** TCP delivers bytes in order, so one missing segment holds up everything behind it, even frames that belong to other objects.
- As in HTTP/1.1, browsers have an **incentive to open multiple parallel TCP connections** to reduce stalling and increase the overall throughput.
- There is **no security over a vanilla TCP connection**.

HTTP/3 addresses these: it **adds security** and **per-object error and congestion control (more pipelining)** and runs over **UDP** instead of TCP.

Daily-life example

HTTP/2 over one TCP connection is like sending 4 different parcels in one single delivery truck: if one box falls off the truck and the driver has to go back to look for it, all four parcels are delayed. HTTP/3 gives each parcel its own tracking, so a problem with one parcel does not hold up the others.

3.14 QUIC: Quick UDP Internet Connections

Slides 50-53

QUIC is an **application-layer protocol on top of UDP**. It was created to **increase the performance of HTTP** and is deployed on many Google servers and apps (the Chrome browser and the mobile YouTube app). It is what HTTP/3 uses.

The protocol stacks compared (slide 50)

```
HTTP/2 over TCP                    HTTP/3 (HTTP/2 over QUIC over UDP)
---------------                    ----------------------------------
Application:  HTTP/2               Application:  HTTP/3  (HTTP/2 slimmed down)
              TLS                                QUIC    (security + loss control)
Transport:    TCP                  Transport:    UDP
Network:      IP                   Network:      IP
```

In the first stack, TLS (security) is a separate layer above TCP (reliability, congestion control). In the QUIC stack, QUIC itself provides reliability, congestion control and security on top of plain UDP. QUIC **adopts approaches that we will study in the transport chapter** for connection establishment, error control and congestion control. The QUIC specification itself says that readers familiar with TCP's loss detection and congestion control will find algorithms in it that parallel the

well-known TCP ones: QUIC re-implements the good ideas of TCP, but in the application layer, where they are easy to update.

Connection establishment (slides 51-52)

- **TCP + TLS** (the HTTP/2 way): first a **TCP handshake** (transport layer, giving reliability and congestion control state), and only afterwards a **TLS handshake** (security: authentication and crypto state). That is **two serial handshakes**, each costing round trips, before the first byte of data can be sent.
- **QUIC: one handshake** establishes reliability, congestion control, authentication and crypto state all together, in **1 RTT**. Then the data flows.

0-RTT connection establishment (slide 53)

QUIC can go even faster for a repeat visit. During the **first QUIC connection**, the server sends a **TLS session ticket** which the client **caches**. When the client later starts a **second QUIC connection**, it **uses the last session ticket** for encryption and authentication, so it can send its data **immediately in the first packet**: a **0 RTT handshake delay**.

> **Daily-life example**
>
> Joining a gym. On your first day you must stand in one queue for registration (TCP handshake) and then in another for ID verification and getting your locker (TLS handshake): two serial queues. QUIC merges the two into a single desk. From the second day onwards you flash your membership card (session ticket) at the gate and walk straight in without any desk visit: that is 0-RTT.

Part 4: E-mail: SMTP, IMAP

4.1 The three major components of e-mail

Slides 55-56

The e-mail system has three major components:

1. **User agents**
2. **Mail servers**
3. **Simple Mail Transfer Protocol: SMTP**

User agent (UA)

- Also known as a **'mail reader'**.
- It is used for **composing, editing and reading** mail messages.
- Examples: Outlook, the iPhone mail client.
- **Outgoing and incoming messages are stored on the server**, not on your device alone.

Mail servers

- The **mailbox** contains the **incoming** messages for the user.
- There is a **message queue** of **outgoing** (to be sent) mail messages. If the receiving server is temporarily unreachable, the message waits in the queue to be tried again.
- The **SMTP protocol** is used **between mail servers** to send e-mail messages. In this exchange the **sending mail server acts as the 'client'** and the **receiving mail server acts as the 'server'**.

> **Daily-life example**
>
> The **post office system**. Your **user agent** is you writing the letter at home. Your **mail server** is your local post office: it has an 'outgoing tray' (message queue) and a set of **PO boxes** (user mailboxes) for incoming letters. **SMTP** is the truck service that carries letters between post offices. The letter always goes: you, to your post office, to your friend's post office, into your friend's PO box, and finally to your friend when he comes to collect it.

4.2 SMTP (RFC 5321)

Slide 57

- SMTP uses **TCP** to reliably transfer an e-mail message from the client (the mail server initiating the connection) to the server, on **port 25**.
- **Direct transfer:** the sending server (acting like a client) connects directly to the receiving server. There are no intermediate mail servers in between.
- **Three phases of transfer:** (1) **SMTP handshaking (greeting)**, (2) **SMTP transfer of messages**, (3) **SMTP closure**.
- **Command/response interaction** (like HTTP): **commands** are ASCII text; the **response** is a status code and phrase.

The timeline on the slide: the client initiates the TCP connection (one RTT), the server answers with **220** (service ready), the client says **HELO**, the server replies **250 Hello**, and then the SMTP transfers begin.

Illustrative SMTP conversation (an extra example beyond the slide; it follows the standard textbook example)

```
S: 220 hamburger.edu                                  (server: ready)
C: HELO crepes.fr                                     (client says hello)
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>                       (who is sending)
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>                       (who is receiving)
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA                                               (here comes the message)
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .                                                  (a single dot ends the message)
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

It reads like a polite human dialogue: greet, say who is sending and who is receiving, send the text, end it with a dot and say goodbye. Every server answer starts with a numeric code that a program can easily understand (2xx means OK, 3xx means 'go on', 5xx means an error).

4.3 Scenario: Alice sends an e-mail to Bob

Slide 58

1. Alice uses her **user agent (UA)** to compose an e-mail message addressed 'to' bob@someschool.edu.
2. Alice's UA sends the message to **her mail server using SMTP**; the message is placed in the **message queue**.
3. The **client side of SMTP** at Alice's mail server opens a **TCP connection** with Bob's mail server.
4. The SMTP client **sends Alice's message over the TCP connection**.
5. Bob's mail server **places the message in Bob's mailbox**.
6. Bob invokes his user agent to **read the message**.

Key idea

Notice that the message goes **directly from Alice's mail server to Bob's mail server**, and that it is pushed by SMTP (the sender initiates), whereas Bob has to *pull* the message to his own device in step 6. That 'pull' is done by a different protocol (next section).

4.4 Retrieving e-mail: mail access protocols

Slide 59

SMTP is only for **delivery and storage of e-mail messages on the receiver's server**. When Bob wants to **retrieve** his mail from his mail server onto his device, he needs a separate **mail access protocol**:

- **IMAP (Internet Mail Access Protocol, RFC 3501):** the messages stay **stored on the server**, and IMAP provides **retrieval, deletion and folders** of the stored messages on the server. (So you see the same inbox and folders from your phone and from your laptop.)
- **HTTP:** webmail services such as Gmail, Hotmail and Yahoo! Mail provide a **web-based interface** on top of SMTP (to send) and IMAP (or POP) to retrieve messages.

The overall picture: sender's UA to sender's mail server uses **SMTP**; sender's mail server to receiver's mail server uses **SMTP**; receiver's mail server to receiver's UA uses a **mail access protocol (IMAP, HTTP)**.

Daily-life example

SMTP is the postman who **delivers** the letter to your PO box. IMAP is you **going to the PO box** to look at, sort into folders, and throw away your letters, and the originals stay at the post office unless you delete them. This 'push' (SMTP) and 'pull' (IMAP) difference is the heart of the topic. (*Why not use SMTP for retrieval as well?* Because SMTP is a push protocol: the server initiates, but your phone may be switched off, so you must be the one to ask for the mail.)

Common confusion

SMTP and IMAP are **not alternatives**. They do different jobs and are used together. SMTP = sending and server-to-server transfer. IMAP/HTTP/POP = retrieving from your own mail server.

Part 5: The Domain Name System (DNS)

5.1 What is DNS and why do we need it?

Slide 61

The **Domain Name System (DNS)** is:

- A **distributed database** implemented in a **hierarchy of many name servers**, and
- An **application-layer protocol**: hosts and DNS servers communicate to **resolve names** (that is, to perform address/name translation).

The slide notes that DNS is a **core Internet function** yet it is implemented as an application-layer protocol: this is the principle of **complexity at the network's 'edge'**, meaning that the network core stays simple and the clever stuff is done in end systems.

The problem DNS solves

People have many identifiers (SSN, name, passport number). In the same way **Internet hosts and routers** have two kinds of identifiers:

- **IP address** (32 bit), which is used for addressing datagrams (machines like numbers), and
- a **name**, for example cs.umass.edu, which is used by **humans** (people like names).

Question asked on the slide

Q: How do we map between an IP address and a name, and vice versa?

A: Using DNS.

Daily-life example

The **contacts list of your phone**. You tap 'Ahmed' and the phone dials 0300-xxxxxxx. You never remember the number, only the name. DNS is the Internet's contact list. Another example closer to home: you know a person by his name, but government offices identify him by his CNIC number. Machines (routers) identify hosts by IP address, humans by name, so a translation service must exist.

5.2 DNS services and why DNS is not centralized

Slide 62

DNS services

- **Hostname-to-IP-address translation** (the main service).
- **Host aliasing:** a host with a complicated **canonical name** can have one or more easy **alias names**.
- **Mail server aliasing:** so that your mail address can be bob@company.com even if the real mail server has a complicated name.
- **Load distribution:** for **replicated Web servers**, a set of **many IP addresses correspond to one name**. The DNS answer can rotate among them to spread the load.

Question asked on the slide

Q: Why not centralize DNS (one giant server)?

A: Because it **does not scale**: it would be a **single point of failure**; it would handle an enormous **traffic volume**; the **centralized database would be distant** from many clients; and **maintenance** of such a giant database would be a nightmare. For scale: Comcast's DNS servers alone handle about **600 billion** DNS queries per day and Akamai's DNS servers alone about **2.2 trillion** per day.

Daily-life example

Imagine **a single telephone-enquiry office in Islamabad** for the entire country. Everybody in Faisalabad, Karachi and Peshawar calls the same office to ask for numbers. The lines would be permanently busy, the office is far away for most callers, and if it burns down nobody can call anyone. A distributed system is like each city having its own directory, with a clear structure for knowing whom to ask.

5.3 Thinking about DNS

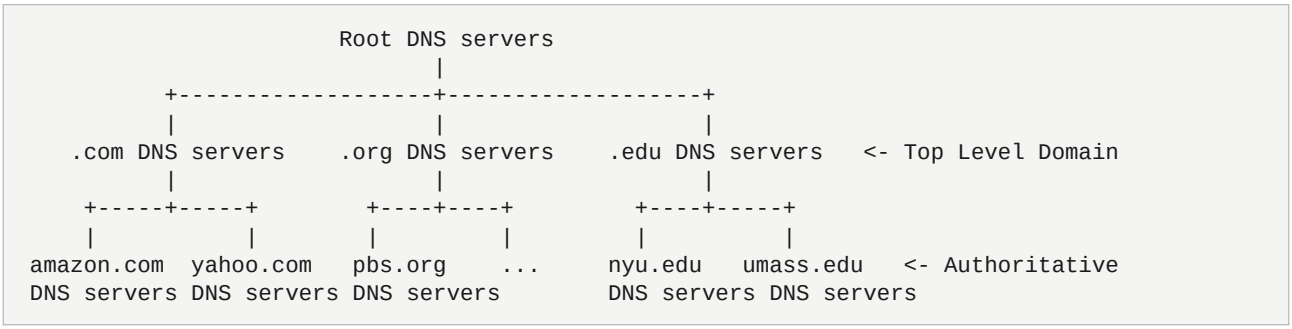
Slide 63

- A **humongous distributed database**: roughly a billion records, each **simple**.
- It handles **many trillions of queries per day**, with **many more reads than writes** (looking up is frequent, changing is rare).
- **Performance matters:** almost every Internet transaction interacts with DNS, so **milliseconds count**.
- **Organizationally and physically decentralized:** millions of different organizations are responsible for their own records.
- It must be **'bulletproof'**: reliable and secure.

5.4 DNS: a distributed, hierarchical database

Slide 64

DNS is organized as a **tree with three main levels**:



- **Root DNS servers** at the top.
- **Top-Level Domain (TLD) servers** in the middle: the .com servers, .org servers, .edu servers, etc.

- **Authoritative DNS servers** at the bottom: the servers of each organization (amazon.com, pbs.org, nyu.edu, umass.edu, yahoo.com...).

First approximation: a client wants the IP address for **www.amazon.com**:

1. The client queries a **root server** to find the **.com DNS server**.
2. The client queries the **.com DNS server** to get the **amazon.com DNS server**.
3. The client queries the **amazon.com DNS server** to get the **IP address for www.amazon.com**.

Daily-life example

Finding a friend's house in a country. First you ask 'which country?' (root points to the right TLD), then 'which city?' (the TLD points to the organization), and finally the local resident of that street tells you the exact house (the authoritative server gives the exact address). Notice also that a domain name reads from **right to left in importance**: www.amazon.**com**: the last part (.com) is the TLD, amazon is the organization and www is the particular host.

5.5 Root name servers

Slides 65-66

- The root servers are the **official 'contact-of-last-resort'** for name servers that **cannot resolve a name** themselves.
- **ICANN** (Internet Corporation for Assigned Names and Numbers) **manages the root DNS domain**.
- It is an **incredibly important Internet function**: the Internet could not function without it.
- **DNSSEC** provides security (authentication and message integrity).
- There are **13 logical root name 'servers'** worldwide, and each 'server' is **replicated many times** (hundreds of physical machines in many countries). The slide's map shows their locations.

5.6 TLD servers and authoritative servers

Slide 67

Top-Level Domain (TLD) servers

- Responsible for **.com, .org, .net, .edu, .aero, .jobs, .museums** and for all **top-level country domains**, for example **.cn, .uk, .fr, .ca, .jp** (and, of course, .pk for Pakistan).
- **Network Solutions** is the authoritative registry for the **.com and .net** TLD; **Educause** for **.edu**.

Authoritative DNS servers

- An **organization's own DNS server(s)**, providing the **authoritative hostname-to-IP mappings** for that organization's named hosts (web server, mail server...).
- They can be maintained **by the organization itself or by a service provider**.

Authoritative means 'the final official source of truth for this name'. Other servers only know the answer second-hand.

5.7 Local DNS name servers

Slide 68

- When a host makes a DNS query, it is sent to its **local DNS server** first.
- The local DNS server replies either **from its local cache** of recent name-to-address translation pairs (which may be **out of date**!), or by **forwarding the request into the DNS hierarchy** for resolution.
- **Each ISP has a local DNS name server.** The local DNS server does **not strictly belong to the hierarchy**.

The slide shows how to find yours: on macOS *scutil --dns*, on Windows *ipconfig /all*. On EndeavourOS you can use:

```
resolvectl status              # if systemd-resolved is in use
cat /etc/resolv.conf           # the resolver your system is configured with
nmcli dev show | grep DNS      # DNS servers handed out by NetworkManager
```

Daily-life example

Your **local DNS server is like the receptionist of your office**. You never call the outside world yourself; you ask the receptionist, and he either answers immediately from his own notebook (cache) or calls the right people on your behalf, and writes the answer in his notebook for the next time.

5.8 DNS name resolution: iterated vs recursive queries

Slides 69-70

The slides use one example: a host at **engineering.nyu.edu** wants the IP address of **gaia.cs.umass.edu**. Its local DNS server is dns.nyu.edu; the authoritative server for umass is dns.cs.umass.edu.

Iterated query (slide 69)

The contacted server replies with the name of the server to contact: 'I don't know this name, but ask this server.' The local DNS server does all the legwork:

1. The requesting host asks its **local DNS server** (dns.nyu.edu) for gaia.cs.umass.edu.
2. The local DNS server asks the **root DNS server**.
3. The root replies: 'I don't know, ask the **.edu TLD server**' (it gives its address).
4. The local DNS server asks the **TLD DNS server**.
5. The TLD replies: 'ask the **authoritative server dns.cs.umass.edu**'.
6. The local DNS server asks the **authoritative DNS server**.
7. The authoritative server replies with the **IP address** of gaia.cs.umass.edu.
8. The local DNS server returns the answer to the requesting host.

Recursive query (slide 70)

A **recursive query puts the burden of name resolution on the contacted name server**: each server goes and asks the next one on your behalf and passes the final answer back down the chain:

1. The host asks the local DNS server.
2. The local DNS server asks the root DNS server.
3. The root asks the TLD server.
4. The TLD asks the authoritative server.
5. The authoritative server answers the TLD with the IP address.
6. The TLD answers the root.
7. The root answers the local DNS server.
8. The local DNS server answers the host.

The slide asks about the drawback: **heavy load at the upper levels of the hierarchy?** Yes: with recursion the root and TLD servers have to remember and handle every request chain, which is why in practice the root and TLD servers prefer iterated queries. (*Extra:* in the typical real-world arrangement, the host's query to its local DNS server is recursive and the local DNS server then uses iterated queries towards the hierarchy.)

Daily-life example

Iterated: You ask a shopkeeper for directions to a house. He says, 'I don't know, but ask the tea-stall owner at the corner.' You walk there and ask him. He says, 'Ask the pharmacist.' You walk there. Finally the pharmacist shows you the house. You did all the walking. **Recursive:** You tell your assistant the name. He asks his friend, who asks his friend, who finds the answer. The answer is passed back through the same chain to you. You did no walking, but the people in the middle were busy.

5.9 Caching DNS information

Slide 71

- Once **any name server learns a mapping, it caches the mapping** and immediately returns the cached mapping in response to a query.
- **Caching improves the response time** (no need to climb the hierarchy again).
- Cache entries **time out (disappear) after some time, the TTL** (time to live).
- **TLD servers are typically cached in the local name servers**, so the root servers are visited rarely.
- **Cached entries may be out of date.** If a named host changes its IP address, it may not be known Internet-wide until **all the TTLs expire**.
- Therefore DNS translation is **best-effort**.

Daily-life example

You saved a friend's number in your phone. If he changes his number, you keep dialling the old one until you learn about the change and update your contacts. The TTL is like the **expiry date on a milk packet**: after the TTL the cached answer is thrown away and the question is asked again. A short TTL gives fresher answers but more queries; a long TTL gives fewer queries but stale answers.

5.10 DNS records

Slide 72

The DNS database stores **resource records (RR)** in the format **(name, value, type, ttl)**. The meaning of name and value depends on the type:

- **type = A: name** is a hostname, **value** is its **IP address**. (This is the basic hostname-to-IP mapping.)
- **type = CNAME: name** is an **alias name** for some 'canonical' (the real) name; **value** is the canonical name. Example from the slide: **www.ibm.com** is really **servereast.backup2.ibm.com**.
- **type = NS: name** is a **domain** (e.g., foo.com); **value** is the **hostname of the authoritative name server** for this domain. (It is what lets a TLD point you to the right authoritative server.)
- **type = MX: value** is the name of the **SMTP mail server** associated with the name (this is how an e-mail sent to user@foo.com finds the mail server of foo.com).

> **Daily-life example**
>
> Think of an **address book with different kinds of entries**. **A** = 'Name to phone number'. **CNAME** = 'nickname to real name' ('Bhai' is really 'Ahmed Khan'). **NS** = 'for this department, the person in charge of the directory is X'. **MX** = 'this company's incoming post is handled at counter Y'.

5.11 DNS protocol messages

Slides 73-74

DNS **query and reply messages have the same format**. The message has a header and four variable-length sections:

```
|<------- 2 bytes ------->|<------- 2 bytes ------->|
|      identification     |          flags          |
|      # questions        |      # answer RRs       |
|      # authority RRs    |    # additional RRs     |
|   questions       (variable number of questions)          |
|   answers         (variable number of RRs)                |
|   authority       (variable number of RRs)                |
|   additional info (variable number of RRs)                |
```

The header (slide 73)

- **Identification:** a 16-bit number for the query; the **reply to the query uses the same number**, so the host can match answers to its questions.
- **Flags:** whether the message is a **query or a reply**, **recursion desired**, **recursion available** and **reply is authoritative**.
- The four counts say how many entries there are in the sections below.

The body (slide 74)

- **Questions:** the **name and type fields for a query** (what is being asked).
- **Answers:** the **RRs in response to the query**.
- **Authority: records for authoritative servers**.

- **Additional info:** extra **'helpful'** information that may be used (for example, the IP address of the authoritative server named in the previous section, so you need not ask again).

> **Daily-life example**
> A **request form with a serial number**. You write your question on the form with serial number 4021. The reply form comes back with the same serial number 4021, so you know which of your several questions it answers. The answer, the 'who is the official authority' note and a 'by the way, here is their phone number' note are all on the same form.

5.12 Getting your info into the DNS

Slide 75

Example: a new startup, **'Network Utopia'** (the slide spells the name networkutopia.com):

1. **Register the name** networkutopia.com at a **DNS registrar** (e.g., Network Solutions).
2. Provide the **names and IP addresses of the authoritative name server** (primary and secondary).
3. The registrar **inserts two RRs into the .com TLD server**: *(networkutopia.com, dns1.networkutopia.com, NS)* and *(dns1.networkutopia.com, 212.212.212.1, A)*.
4. Create a **local authoritative server** with the IP address 212.212.212.1. In it, put a **type A record for www.networkutopia.com** and a **type MX record for networkutopia.com**.

> **Daily-life example**
> Opening a new shop in a city. You register your shop's name at the city directory office (the registrar/TLD): 'for any question about this shop, contact this person at this address' (the NS and A records). Then you personally keep the detailed catalogue at the shop (authoritative server) with the actual information such as the web server's address and the mail server.

5.13 DNS security

Slide 76

DDoS (Distributed Denial of Service) attacks

- **Bombard the root servers** with traffic: **not successful to date**, because of **traffic filtering** and because **local DNS servers cache the IPs of TLD servers**, which allows the root servers to be bypassed.
- **Bombard TLD servers: potentially more dangerous**, since they are not so easily bypassed.

Spoofing attacks

- **Intercept DNS queries** and return **bogus replies**.
- **DNS cache poisoning:** fill a DNS cache with false mappings so that later users get wrong addresses.

The defence mentioned is **DNSSEC (RFC 4033)**, which provides **authentication services** so that you can verify that a DNS answer is genuine and unchanged.

Daily-life example

Spoofing is like someone **secretly replacing a page in the phone directory** so that when you look up your bank, you are given the number of a fraudster. DNSSEC adds a **signature/seal** to every directory page so that tampering is detected. A DDoS attack is like thousands of people calling the enquiry office at once so that real callers cannot get through.

Part 6: Video Streaming and Content Distribution Networks

6.1 Context: why video is special

Slide 78

- **Streaming video traffic is a major consumer of Internet bandwidth.** Netflix, YouTube and Amazon Prime together accounted for about **80% of residential ISP traffic** (2020 figure on the slide).
- **Challenge 1: scale.** How do you reach about **1 billion users**?
- **Challenge 2: heterogeneity.** Different users have different capabilities: wired versus mobile, bandwidth-rich versus bandwidth-poor.
- **Solution:** a **distributed, application-level infrastructure** (the CDN, shown below).

6.2 Multimedia: video and how it is compressed

Slide 79

A **video** is a **sequence of images displayed at a constant rate**, for example **24 images per second**. A **digital image** is an **array of pixels**, and each pixel is represented by bits. Raw video would need an enormous number of bits, so we use **coding**: using **redundancy** (repeated information) within and between images to decrease the number of bits needed.

- **Spatial coding (within one image):** instead of sending N values of the same colour (say, N purple pixels in a row), send only **two values: the colour (purple) and the number of repeats (N)**.
- **Temporal coding (from one image to the next):** instead of sending the complete frame i+1, send only the **differences from frame i**.

Daily-life example

Spatial: if you must describe a photo of a clear sky over the phone, you would not say 'blue pixel, blue pixel, blue pixel...' 5000 times; you would say '5000 blue pixels'. **Temporal:** in a news video where only the presenter's lips move and the studio behind him stays the same, each new frame can say 'same as before, except the mouth region'. A **flipbook** artist who redraws only the moving character on each page, and not the whole background, is doing temporal coding.

6.3 Content Distribution Networks (CDNs)

Slides 80-82

The challenge: how do we stream content (selected from **millions of videos**) to **hundreds of thousands of simultaneous users**?

Option 1: a single large 'mega-server'

- **Single point of failure** (if it fails, everybody loses service).
- **Point of network congestion** (all traffic is funnelled into one place).

- **Long (and possibly congested) path to distant clients.**

The slide's verdict: **this solution simply does not scale.**

Option 2: store/serve multiple copies at multiple geographically distributed sites (a CDN)

The two main philosophies for placing the servers:

- **Enter deep:** push CDN servers **deep into many access networks**, **close to users**. Used by **Akamai**, which had about **240,000 servers deployed in more than 120 countries** (2015 figure).
- **Bring home:** a **smaller number (tens) of larger clusters** placed in **POPs (points of presence) near access networks**. Used by **Limelight**.

Slide 82 ('Akamai today') shows a figure from the source cited on the slide about Akamai's present-day global footprint. The slide note adds: Akamai has 100,000+ servers in 1000+ clusters in 1000+ networks in 70+ countries, serving trillions of requests a day. The point to remember: **CDNs are huge, worldwide infrastructures.**

Daily-life example

A **fast-food chain**. If a company made all burgers in one giant kitchen in Karachi and delivered worldwide, burgers would be late, cold, and the single kitchen would be jammed with orders and would be a disaster if it caught fire (mega-server). So the chain opens **branches in every city** and stocks each with the popular items: customers are served from the nearest branch (CDN servers close to users). 'Enter deep' is branches in every neighbourhood; 'bring home' is a few large regional kitchens.

6.4 How does Netflix work?

Slide 83

1. Netflix **stores copies of its content** (for example the series MADMEN) at its worldwide **OpenConnect CDN nodes**.
2. The **subscriber requests content** (for example, clicks on Madmen on the Netflix site); the **service provider returns a manifest file** (a list of where the copies are and in which versions).
3. **Using the manifest, the client retrieves the content at the highest supportable rate.**
4. It **may choose a different rate or a different copy** if the network path is congested.

Daily-life example

The manifest is like the **'branch list' on a restaurant app**: you open the menu, you choose a dish, and the app gives you the list of the nearest branches that have it, with the available portion sizes. Your phone picks the best one; when the road to the first branch gets blocked in traffic, it quietly switches to another branch or a smaller (lower-quality) portion so that you still get your video without constant pauses. This switching of quality while watching is the idea of **DASH** mentioned earlier.

Part 7: Socket Programming

7.1 Goal and the two socket types

Slides 84-85

Goal: learn how to build **client/server applications that communicate using sockets**. A **socket is the door between the application process and the end-to-end transport protocol** (the same figure as slide 9: the part above the socket is controlled by the application developer, the part below by the operating system).

There are **two socket types for the two transport services**:

- **UDP: unreliable datagram**. Each message is an independent package.
- **TCP: reliable, byte stream-oriented**. A continuous pipe of bytes.

The example application used in the slides

1. The client reads a **line of characters** (data) from its keyboard and sends the data to the server.
2. The server receives the data and **converts the characters to uppercase**.
3. The server sends the modified data to the client.
4. The client receives the modified data and **displays the line on its screen**.

7.2 Client/server socket interaction: UDP

Slide 86

The flow chart on slide 86 shows the order of calls on each side:

Server (running on serverIP)

1. **Create a socket** on port x: *serverSocket = socket(AF_INET, SOCK_DGRAM)*. (AF_INET means 'IPv4 addressing'; SOCK_DGRAM means 'datagram', i.e., UDP.)
2. **Read the datagram from serverSocket** (and learn who sent it).
3. **Write the reply to serverSocket, specifying the client's address and port number.**

Client

1. **Create a socket**: *clientSocket = socket(AF_INET, SOCK_DGRAM)*.
2. **Create a datagram with the serverIP address and port = x**, and **send the datagram via clientSocket**.
3. **Read the datagram from clientSocket** (the reply).
4. **Close clientSocket.**

> **Key idea**
>
> In UDP there is **no handshake and no connection**. Every datagram that is sent must carry the destination IP address and port, and the receiver learns the sender's address from each arriving datagram so that it can reply.

Daily-life example

UDP sockets are postcards. Every postcard carries the full address of the recipient, you drop it in the box and you do not know whether it arrives. The recipient reads the sender's address on it and writes back another postcard with that address. **TCP sockets are a phone call:** first one side dials and the other picks up (a connection), after which both only speak into the phone, without repeating the address in every sentence.

Python code for the example (extra, to help you practise)

The slides only show the flowchart. Below is the same uppercase application written in Python (you already know Python, so this makes the sockets concrete). Run the server in one terminal and the client in another.

UDPServer.py

```
# UDPServer.py
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)    # create a UDP socket
serverSocket.bind(('', serverPort))           # attach port 12000 to this socket
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)   # datagram + sender address
    modifiedMessage = message.decode().upper()             # convert to uppercase
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)  # reply to that client
```

UDPClient.py

```
# UDPClient.py
from socket import *

serverName = 'localhost'                      # or the server's IP / hostname
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)    # create a UDP socket
message = input('Input lowercase sentence: ')
clientSocket.sendto(message.encode(), (serverName, serverPort))  # address goes with it
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)     # wait for the reply
print(modifiedMessage.decode())
clientSocket.close()
```

TCP version (for comparison; the slide mentions the TCP socket but the figure is only for UDP). The main differences are that the client must **connect** first (the TCP handshake happens there) and that the server has a **welcoming socket** which creates a **new connection socket for each client**:

TCPServer.py and TCPClient.py

```
# TCPServer.py
from socket import *
serverSocket = socket(AF_INET, SOCK_STREAM)   # SOCK_STREAM = TCP
serverSocket.bind(('', 12000))
serverSocket.listen(1)                        # welcoming socket: wait for connections
while True:
    connectionSocket, addr = serverSocket.accept()    # new socket just for this client
    sentence = connectionSocket.recv(1024).decode()
    connectionSocket.send(sentence.upper().encode())
    connectionSocket.close()

# TCPClient.py
from socket import *
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect(('localhost', 12000))     # TCP handshake happens here
clientSocket.send(input('Input lowercase sentence: ').encode())
print(clientSocket.recv(1024).decode())        # no address needed any more
clientSocket.close()
```

Part 8: Chapter Summary and Revision Aids

8.1 What the chapter covered

Slides 87-88

- **Specific protocols:** HTTP, SMTP, IMAP, DNS, P2P (BitTorrent), video streaming and CDNs.
- **Socket programming:** TCP and UDP sockets.
- **Application architectures:** client-server and P2P.
- **Application service requirements:** reliability, bandwidth, delay.
- **The Internet transport service model:** connection-oriented, reliable service (TCP) and unreliable datagrams (UDP).

The most important thing, as the last slide says, is that you **learned about protocols**. Typical protocol themes that appeared again and again:

- **Typical client/server request/reply message exchange:** the client asks, the server answers (HTTP, SMTP, DNS).
- **Centralized vs decentralized:** a single mega-server (centralized) versus DNS, CDNs and P2P (distributed).
- **Stateless vs stateful:** HTTP is stateless (simple) and cookies add state on top.
- **Scalability:** caching, DNS hierarchy, CDNs and P2P all exist so that the system keeps working when millions of users come.
- **Reliable vs unreliable message transfer:** TCP vs UDP, and which applications need which.
- **'Complexity at network edge':** the Internet core is simple; applications such as the Web, mail and DNS are implemented in the end systems.

8.2 Common confusions, cleared

Common confusion

Client/server architecture vs client/server process. The architecture is a design of the whole application. A client or server *process* is a role in a single conversation, and a P2P peer plays both roles.

Common confusion

IP address vs port number. The IP finds the host (the building), the port finds the process (the flat).

Common confusion

TCP vs UDP. TCP: reliable, flow control, congestion control, connection setup, but no timing guarantee. UDP: none of these, so it is fast and simple. Neither of them provides security by itself; TLS adds it.

Common confusion

Stateless vs cookies. HTTP remains stateless. The cookie technology is built *on top*: the server stores the state in a database and the browser carries the ID.

Common confusion

Non-persistent vs persistent HTTP. Non-persistent: one object per TCP connection, 2 RTT + transmission time per object. Persistent: many objects over one connection.

Common confusion

First-party vs third-party cookies. First-party: set by the site you visit. Third-party: set by another site whose content (an ad) is embedded in the page you visit; used for tracking across sites.

Common confusion

Web cache vs cookie vs CDN. A cache stores copies of objects close to the client to avoid asking the origin server. A cookie remembers who you are. A CDN is a provider's network of many servers storing copies of content around the world.

Common confusion

SMTP vs IMAP. SMTP pushes mail to servers (sending). IMAP lets the user's agent pull mail from its own server (reading).

Common confusion

Iterated vs recursive DNS query. Iterated: the server says 'ask that one' and the asker goes. Recursive: the server resolves it fully on behalf of the asker.

Common confusion

Root vs TLD vs authoritative vs local DNS server. Root: points to TLD. TLD: points to the organization's server. Authoritative: gives the final IP. Local: the one your host asks first; it caches and is not really part of the hierarchy.

8.3 Formulas and numbers to remember

- **RTT** = time for a small packet to go from client to server and back.
- **Non-persistent HTTP, time per object** = 2 RTT + file transmission time.
- **Persistent HTTP:** as little as 1 RTT for all the referenced objects (after the connection is set up).
- **Well-known ports in the slides:** HTTP = 80, SMTP = 25.
- **IP (v4) address** = 32 bits.
- **DNS RR format** = (name, value, type, ttl); types A, CNAME, NS, MX.
- **Status codes:** 200 OK, 301 Moved Permanently, 400 Bad Request, 404 Not Found, 505 HTTP Version Not Supported; 304 Not Modified (conditional GET).
- **HTTP evolution:** HTTP/1.1 (persistent, pipelining, HOL blocking) leads to HTTP/2 (frames, priorities, server push) leads to HTTP/3 (QUIC over UDP).
- **QUIC:** 1-RTT handshake the first time, 0-RTT with a cached session ticket.
- **13** logical root name servers.

8.4 Self-test questions

Try answering these without looking at the notes; the answers are all in the sections above.

1. Compare client-server and P2P architectures. Which has self-scalability and why?
2. What is a socket? Who controls what on each side of the socket?
3. Why does an IP address alone not identify a process? What identifies it?
4. List the four things an application-layer protocol defines. What is the difference between an open and a proprietary protocol?
5. Explain throughput, data integrity, timing and security requirements with one application each.
6. What does TCP provide and what does UDP not provide? Why does UDP still exist?
7. Why is HTTP called stateless? How do cookies add state? Name the four components of cookie technology.
8. Explain how a third-party cookie lets AdX show socks advertisements on the NY Times page.
9. Calculate the time to fetch a page with 1 base file and 10 images using non-persistent and persistent HTTP if the RTT is 80 ms (ignore the transmission time).
10. What is a conditional GET and what does the status code 304 mean?
11. What is head-of-line blocking? How does HTTP/2 reduce it? Why is HTTP/3 over QUIC needed?
12. Describe the path of an e-mail from Alice to Bob. Where are SMTP and IMAP used?
13. Why is DNS distributed? Describe root, TLD, authoritative and local DNS servers.
14. Compare iterated and recursive DNS queries with the help of a diagram.
15. What is TTL in DNS caching? Why can a cached answer be wrong?
16. What are the A, CNAME, NS and MX records?
17. Why is a single mega-server a bad idea for streaming video? How does a CDN solve this? What is a manifest file?
18. Draw the client/server interaction of the UDP uppercase application.

8.5 Glossary

Application layer: The top layer of the Internet stack, where network applications and their protocols live.

Authoritative DNS server: An organization's own DNS server that holds the official name-to-IP mappings for its hosts.

Cache hit / miss: A hit means the cache already had the object; a miss means it had to fetch it from the origin server.

CDN: Content Distribution Network: servers spread around the world that store copies of content close to users.

Client / server process: The process that starts a communication / the process that waits to be contacted.

Conditional GET: A GET with If-modified-since; the server returns 304 Not Modified if the cached copy is still fresh.

Cookie: An ID value set by a website, kept by the browser and returned with each request so the site can recognize the user.

DASH: Dynamic Adaptive Streaming over HTTP: the player fetches video chunks at a quality that matches the current network speed.

DNS: Domain Name System: distributed database and protocol that translates host names into IP addresses.

Elastic application: An application that can use whatever throughput it gets (file transfer, e-mail, web).

Flow control: TCP mechanism that stops the sender from overwhelming the receiver.

Congestion control: TCP mechanism that slows the sender when the network is overloaded.

GDPR: EU General Data Protection Regulation; treats identifying cookies as personal data and gives users control.

HOL blocking: Head-of-line blocking: a small object waits behind a large one in a first-come-first-served queue.

HTTP: HyperText Transfer Protocol: the Web's application-layer protocol (request/response).

ICANN: The organization that manages the DNS root domain.

IMAP: Internet Mail Access Protocol: lets a user agent retrieve, delete and organize mail stored on the mail server.

IP address: 32-bit (IPv4) number identifying a host.

Manifest file: A file returned by the video service listing where and in which versions the content can be fetched.

Non-persistent / persistent HTTP: One object per TCP connection / many objects over one TCP connection.

P2P: Peer-to-peer: end systems communicate directly and both request and provide service.

Port number: Number that identifies a process on a host (80 for HTTP, 25 for SMTP).

Protocol: A set of rules defining message types, syntax, semantics and the rules of exchanging them.

QUIC: An application-layer protocol over UDP with built-in reliability, congestion control and security; used by HTTP/3.

RFC: Request for Comments: the public documents that define open protocols.

RR (resource record): A DNS entry of the form (name, value, type, ttl).

RTT: Round-Trip Time: time for a small packet to go to the server and come back.

Socket: The door between an application process and the transport protocol.

SMTP: Simple Mail Transfer Protocol: transfers mail between mail servers over TCP, port 25.

Stateless: The server keeps no information about past requests.

TCP / UDP: Reliable connection-oriented transport / unreliable connectionless transport.

TLD server: Top-Level Domain server (.com, .org, .edu, country codes) that points to authoritative servers.

TLS: Transport Layer Security: encrypted, authenticated, integrity-protected connections on top of TCP.

TTL: Time to live: how long a cached DNS entry may be used before it expires.

URL: Address of a web object: host name plus path name.

User agent: The mail reader program (Outlook, phone mail app) used to compose and read messages.

Web cache / proxy: A server that satisfies requests from stored copies and otherwise fetches from the origin server.

Best of luck with your studies! These notes follow the order of the slides, so you can read them side by side with the deck and tick off each slide as you understand it.